

Documentación del Sistema de Trading Automatizado para Binance

Cero

20 de octubre de 2025

Índice

1. Introducción	2
1.1. Objetivos del Proyecto	2
2. Arquitectura del Proyecto	2
3. Configuración Inicial	3
3.1. Prerrequisitos	3
3.2. Instalación de Dependencias	3
3.3. Archivo de Configuración (.env)	3
4. Guía de Uso	3
4.1. Ejecución en Modo Real (Live Trading)	3
4.2. Backtesting de Estrategias	4
4.2.1. Paso 1: Descargar Datos Históricos	4
4.2.2. Paso 2: Ejecutar el Backtest	4
4.3. Visualización con el Dashboard	4
5. Despliegue en Servidor	4
5.1. Uso de <code>screen</code>	4

1. Introducción

Este documento describe la arquitectura, configuración y uso del sistema de trading automatizado diseñado para interactuar con la API de Binance. El sistema está compuesto por un conjunto de scripts de Python que permiten el análisis de mercado en tiempo real, la ejecución de estrategias de trading, el backtesting y la visualización de datos.

1.1. Objetivos del Proyecto

- **Monitorización:** Analizar datos de mercado (precios, volumen) y calcular indicadores técnicos (EMAs, RSI, Bandas de Bollinger, etc.).
- **Detección de Señales:** Identificar patrones de velas y configuraciones estratégicas (Cruce Dorado, Wyckoff, etc.) para generar señales de compra (LONG) y venta (SHORT).
- **Gestión de Riesgo:** Calcular automáticamente los niveles de Stop Loss y Take Profit para cada operación.
- **Notificaciones:** Enviar alertas en tiempo real a través de Telegram.
- **Visualización:** Ofrecer un dashboard interactivo para visualizar gráficos, indicadores y operaciones ejecutadas.
- **Backtesting:** Probar la efectividad de las estrategias utilizando datos históricos.

2. Arquitectura del Proyecto

El sistema se compone de varios scripts de Python, cada uno con una responsabilidad específica:

`common_utils.py` Librería central que contiene funciones compartidas por todos los demás scripts, como la obtención de datos de Binance, el cálculo de indicadores, el registro de operaciones y el envío de notificaciones.

`trading-v6.py` El bot de trading principal. Implementa estrategias basadas en cruces de medias móviles y patrones de velas como Hammer y Shooting Star.

`trading_wyckoff.py` Un bot alternativo que implementa una estrategia simplificada basada en los conceptos de Wyckoff (eventos Spring y Upthrust), además de los patrones de velas.

`dashboard.py` Una aplicación web interactiva construida con Streamlit. Permite visualizar los gráficos de precios, indicadores y las operaciones registradas en `trades_log.csv`.

`download_data.py` Herramienta para descargar grandes volúmenes de datos históricos de Binance y guardarlos en un archivo CSV, esencial para el proceso de backtesting.

`.env` Archivo de configuración para almacenar variables de entorno como claves de API, parámetros de estrategia y tokens de Telegram.

`trades_log.csv` Archivo donde se registran todas las operaciones generadas por los bots, tanto en modo real como en backtesting.

3. Configuración Inicial

3.1. Prerrequisitos

Asegúrate de tener instalado Python 3.8 o superior y pip.

3.2. Instalación de Dependencias

Instala todas las librerías necesarias ejecutando:

```
1 pip install -r requirements.txt
```

3.3. Archivo de Configuración (.env)

Crea un archivo llamado `.env` en la raíz del proyecto con el siguiente contenido y ajústalo a tus necesidades:

```
1 # --- Parámetros Generales ---
2 SYMBOL="BTCUSDT"
3 INTERVAL="1h"
4 LIMIT=250
5 LOG_LEVEL="INFO"
6 SLEEP=3600 # Segundos de espera entre ciclos (1 hora)
7
8 # --- Parámetros de Indicadores ---
9 VOLUME_SMA_PERIOD=20
10 ATR_WINDOW=14
11 BOLLINGER_WINDOW=20
12 POC=65000.0
13
14 # --- Parámetros de Estrategia (trading-v6.py) ---
15 RR_RATIO=2.0
16 SL_BUFFER=0.002
17
18 # --- Parámetros de Estrategia (trading-wyckoff.py) ---
19 RISK_STOP_MULT=1.5
20 RISK_TP_MULT=2.5
21 WYCKOFF="true"
22 WYCKOFF_VOLUME_MULT=1.3
23
24 # --- Logs y Notificaciones ---
25 TRADES_LOG_FILE="trades_log.csv"
26 TELEGRAM_TOKEN="TU_TOKEN_DE_TELEGRAM"
27 TELEGRAM_CHAT_ID="TU_CHAT_ID"
28
29 # --- Backtesting ---
30 BACKTEST="false"
31 BACKTEST_FILE="historical_data.csv"
```

4. Guía de Uso

4.1. Ejecución en Modo Real (Live Trading)

Para ejecutar uno de los bots en modo de operación real, monitorizando el mercado y enviando señales, utiliza los siguientes comandos.

Bot principal (v6):

```
1 python trading-v6.py
```

Bot con estrategia Wyckoff:

```
1 python trading_wyckoff.py --wyckoff
```

4.2. Backtesting de Estrategias

4.2.1. Paso 1: Descargar Datos Históricos

Usa `download_data.py` para obtener los datos necesarios.

```
1 python download_data.py --symbol ETHUSDT --interval 4h --limit 2000 --  
   output eth_4h_data.csv
```

4.2.2. Paso 2: Ejecutar el Backtest

Ejecuta el bot deseado con el flag `--backtest` y especifica el archivo de datos.

```
1 # Para trading-v6.py  
2 python trading-v6.py --backtest --backtest-file eth_4h_data.csv --symbol  
   ETHUSDT  
3  
4 # Para trading_wyckoff.py  
5 python trading_wyckoff.py --backtest --backtest-file eth_4h_data.csv --  
   symbol ETHUSDT
```

4.3. Visualización con el Dashboard

Para iniciar el dashboard interactivo, ejecuta:

```
1 streamlit run dashboard.py
```

Accede a la URL proporcionada en la terminal (normalmente `http://localhost:8501`) para ver los gráficos. El dashboard se actualizará automáticamente y mostrará las operaciones del archivo `trades_log.csv`.

5. Despliegue en Servidor

Para que los scripts se ejecuten de forma continua en un servidor, se recomienda usar `screen` o, para una solución más robusta, un servicio de `systemd`.

5.1. Uso de screen

1. Iniciar una nueva sesión: `screen -S trading-bot`
2. Ejecutar el script dentro de la sesión: `python trading-v6.py`
3. Desprenderse de la sesión: `Ctrl+A` y luego `d`.
4. Para volver a la sesión: `screen -r trading-bot`